

RESEARCH

Open Access



BS-GAT: a network intrusion detection system based on graph neural network for edge computing

Yalu Wang¹ , Zhijie Han², Ying Du^{1,3,4*}, Jie Li⁵ and Xin He²

Abstract

The emergence of Edge Computing has led to an increasingly intricate and widespread issue of network infiltration in Edge Computing devices. Therefore, it is crucial to explore intelligent, automated, and resilient methods for network intrusion detection. Graph neural network-based techniques for network intrusion detection have been proposed by many scholars recently. However, the graph construction methods of these approaches cannot fully adapt to real network intrusion datasets, leading to problems such as overfitting and insufficient graph information mining. Furthermore, the methods employed in the model training phase are relatively limited, failing to consider the characteristic presence of grouping within networks. These shortcomings result in a lack of high accuracy in intrusion detection systems, especially in multi-class classification scenarios. This research suggests a graph neural network technique based on behavior similarity employing a graph attention network (BS-GAT) to handle the aforementioned problem. To address overfitting and inadequate graph information mining, a behavioral similarity-based graph creation method is first presented through an analysis of real datasets. Subsequently, the inclusion of edge behavioral relationship weights into the GAT leverages the relationship between data flow and graph structural information, enhancing the performance of the trained model. In the final stage, experiments were executed using the most recent datasets to assess the effectiveness of the proposed behavior similarity-based graph attention network in network intrusion detection. The findings indicate that the proposed approach is highly effective and significantly outperforms existing solutions. In binary classification, recall, precision, f1-score, and accuracy all exceed 99%. For multi-class performance, the recognition accuracy exceeds 93%, the weighted recall surpasses 91%, and the weighted f1-score exceeds 92%, showing a substantial improvement in multi-class recognition effectiveness.

Keywords Internet of Things, Network intrusion, Graph neural network, Behavior similarity, Graph construction

Introduction

As time progresses, the scale of Internet of Things (IoT) devices has grown significantly. They have become increasingly intricate, encompassing various functional devices such as smart TVs, smartphones, smart speakers, smart water purifiers, routers, smart rice cookers, wireless smoke detectors, wireless printers, and so on. These devices interconnect via edge devices, forming edge networks (Ghasempour 2019). Consequently, the number and frequency of attacks on edge computing device networks have grown exponentially. To facilitate the advancement of the IoT and ensure the security of smart

*Correspondence:

Ying Du
duying@henu.edu.cn

¹ School of Computer and Information Engineering, Henan University, Long Ting District, Kaifeng 475004, Henan, People's Republic of China

² School of Software, Henan University, Long Ting District, Kaifeng 475004, Henan, People's Republic of China

³ Henan Province Engineering Research Center of Spatial Information Processing, Henan University, Kaifeng 475004, People's Republic of China

⁴ Henan Engineering Research Center for Industrial Internet of Things, Zhengzhou 450046, People's Republic of China

⁵ School of Intelligent Engineering, Zhengzhou University of Aeronautics, Zhengzhou 450046, People's Republic of China

networks, researching methods for detecting network intrusions on edge computing devices is of paramount importance.

Network intrusion detection (NID) methods can be broadly categorized into two types: feature-based and machine learning (ML)-based, as outlined in Lo et al. (2022). Feature-based NID systems rely on a predefined set of attack features to analyze and identify suspicious patterns within captured data flows. The quality of the characteristics utilized is a major determinant of this detection method's efficacy. However, it may not be able to respond appropriately to new forms of attacks, which could result in poor detection performance. Modern NID techniques are based on machine learning, with the majority utilizing data flow information directly for identification, as referenced in (He et al. 2019; Lawal et al. 2020; Sarhan et al. 2011; Ahmad et al. 2022; Churcher et al. 2021). While these methods effectively address the feature dependence issue and demonstrate good detection performance against new attacks, their precision in identification falls short for real-world network applications. Recently, researchers have shifted focus towards the emerging field of ML known as graph neural networks (GNNs), introducing several novel NID techniques (Lo et al. 2022; Zhou et al. 2021; Chang and Paula 2021). GNNs are a specialized type of neural network tailored for graph correlation properties. Over the past few years, they have been widely applied across various domains, including social networks (Kipf and Welling 2016), network analysis (Jiang 2021), traffic management (Jiang and Jiayun 2022), medical treatment (Zhang et al. 2021a), and distributed computing (Sun et al. 2021). Given that networks are fundamentally graph-based systems, integrating GNNs into NID techniques is a logical progression that is expected to significantly enhance detection accuracy.

NID methods leveraging GNNs typically involve two primary steps: graph construction and model training. Graph construction entails utilizing available network data and establishing connections to create a logically coherent graph representation. Model training refers to utilizing neural networks on the generated graph to train models used for identifying edges or nodes. The detection performance of NID techniques based on GNNs is better than that of conventional machine learning-based techniques. However, both graph creation and model training provide practical challenges for current NID techniques based on GNNs. For instance, the logic behind graph construction is singular, failing to consider the characteristics of network data flow, which could lead to insufficient graph information mining (Lo et al. 2022; Chang and Paula 2021). Furthermore, the direct application of existing model training techniques neglects the

interdependencies among grouped nodes (Zhou et al. 2021). Consequently, this oversight leads to reduced accuracy in NID methods based on GNNs, particularly in multi-classification scenarios.

To address the aforementioned issues, this paper proposes a Graph Attention Network (GAT) algorithm based on behavioral similarity (BS-GAT) which is used to improve model training as well as graph creation. In the graph construction phase, behavioral rules are represented as edges, and data flow as nodes. This creates a network in which each node has a consistent number of neighbors. This technique effectively addresses overfitting and imperfect graph information mining problems. Weights based on behavioral similarity are introduced in the model training phase to address the node grouping characteristics. By employing these methods and conducting experiments on the latest flow-based datasets, the results show binary classification accuracy exceeding 98% and multi-classification accuracy surpassing 93%, effectively addressing the issue of low accuracy in multi-class scenarios.

To summarize, this paper makes three primary contributions:

- We introduce a distinctive GAT based on behavioral similarity to address the NID problem. By leveraging behavioral similarity, a novel method for graph creation is devised. Data flows are regarded as nodes within the graph, while the behavior rules of these nodes serve as edges, creating a graph where each node has a relatively uniform number of neighbors. This approach effectively addresses issues like uneven graph construction and overfitting prevalent in other GNN-based NID methods.
- We incorporate the weighted edge behavior relationships into the GAT, enabling the aggregation of features from hidden layer nodes. This enhancement allows NID methods to better leverage the relationships between data flows and the structural information within the graph.
- Through experiments using the most recent network flow information, the suggested NID strategy is assessed. The experimental results demonstrate that the proposed method performs better as compared to standard NID methods and graph neural network-based NID techniques.

The remainder of the paper is structured as follows: Sect. "Related work" presents an overview of the advancements made by scholars in the domain of NID methods. Sect. "Behavior similarity based GAT for NID" initially provides an introduction to GNN and GAT, followed by a detailed exposition of the proposed method.

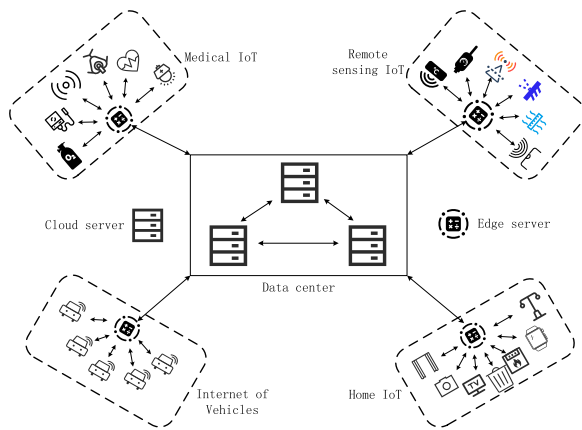


Fig. 1 Cloud-edge collaborative IoT architecture diagram

Sect. "Experimental evaluation" delineates the datasets used, the experimental methodologies employed, and the resulting outcomes. Finally, Sect. "Conclusions" offers a comprehensive summary of the paper.

Related work

Numerous researchers have developed machine learning-based techniques for detecting network intrusions in recent years. From traditional machine learning techniques to the advent of graph-based techniques and ultimately to graph neural network-based techniques, these algorithms have developed over time.

Used a unique hierarchical progressive network structure, He et al. (2019) proposed a multi-modal sequence NID method by extracting different feature levels from network connections, leveraging LSTM and Multi-Modal Deep Autoencoder (MDAE) techniques. In Ahmad

et al. (2022), the authors compare various techniques for developing NID systems and select the optimal features from the dataset based on the correlation between features. Building upon this, they propose an AdaBoost-based NID method and provide a detailed description of its functionality and performance. These approaches train directly on the dataset without taking into account the graph topology information. As a result, each training data point remains relatively isolated, unable to fully exploit the information contained within the dataset. Consequently, these methods exhibit limited efficacy in detecting intricate network attacks, such as Botnet attacks (Vormayr et al. 2017; Bhuyan et al. 2011; Kambourakis et al. 2007), which require a more comprehensive network perspective and traffic analysis. Therefore, seeking a method that can encompass a global network view and traffic for improving the performance of intrusion detection is of significant importance, such as enhancing generalization and stability.

Qingsai et al. (2020) combined statistical features with latent features, transforming network traffic into first-order and second-order graphs. The first-order graph learns latent features from the perspective of individual nodes, while the second-order graph learns latent features from a global perspective. Then, classifier methods were applied to anomaly detection, yielding favorable results on two real datasets. In Islam et al. (2020), the authors proposed a four-stage intrusion detection system that uses the chi-squared method and can detect both strong and weak cyber attacks in a CAN. This work is the first-ever graph-based defense system proposed for the CAN. Despite the fact that both of these techniques employ graph-based techniques to solve intrusion

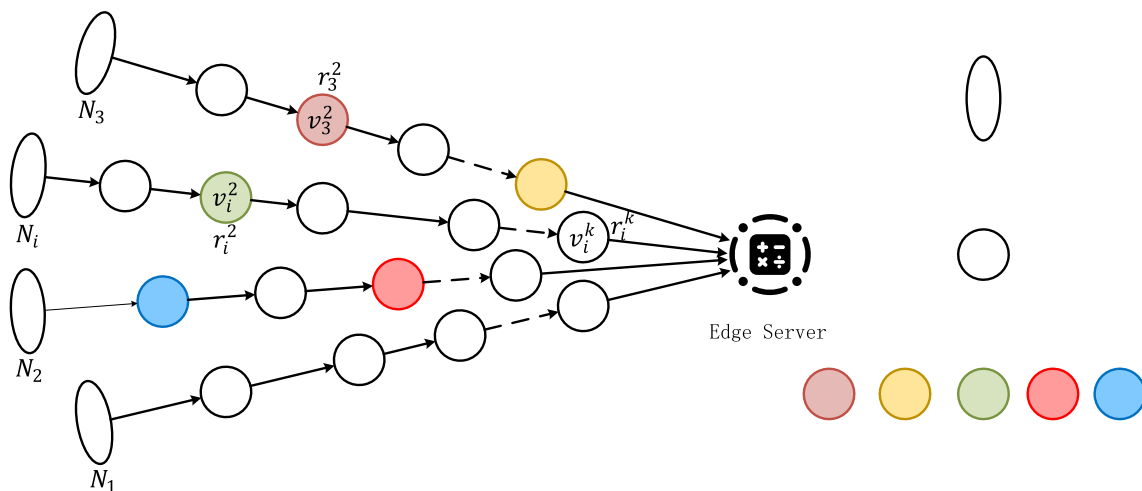


Fig. 2 Problem definition structure

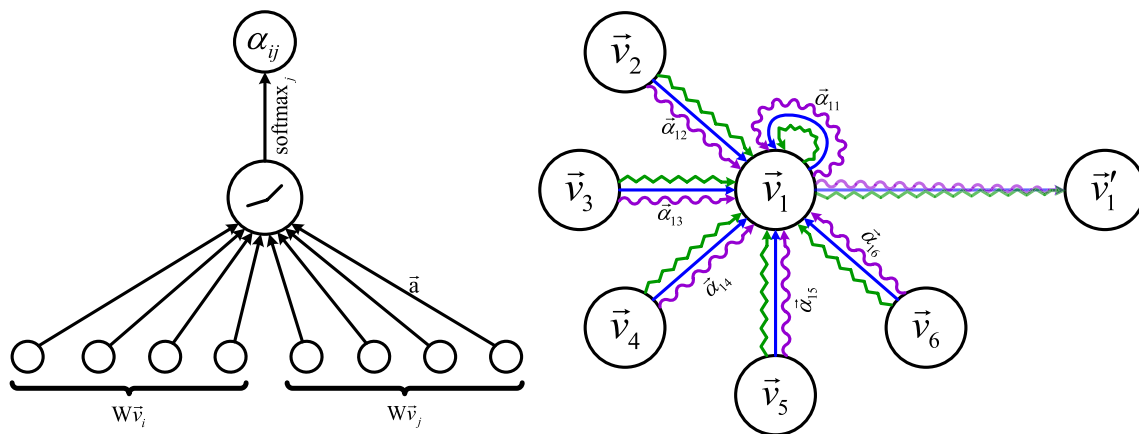


Fig. 3 The diagram illustrates two core computational steps. On the left is the process of computing attention coefficients, while on the right is the process of aggregating hidden layer features

detection issues, they may not fully utilize graph information and have low levels of generalizability.

Due to the remarkable performance of GNN on graph-structured data, numerous scientists have integrated GNN into NID systems. Zhou et al. (2021) proposed a new Hierarchical Adversarial Attack (HAA) generation method for Graph Neural Network (GNN)-based intrusion detection in budget-constrained IoT systems, achieving a hierarchy-aware black-box adversarial attack strategy. Lo et al. (2022) proposed the E-GraphSAGE architecture, a variant of GraphSAGE tailored for capturing both edge features and topology information in graphs for NID within the IoT. Chang et al. (2021) enhanced the E-GraphSAGE architecture by introducing

the E-ResGAT architecture based on Graph Attention Networks (GAT). The primary innovation lies in integrating residual learning into GNN by exploiting available graph information. The three methods, despite utilizing GNNs, suffer from overfitting during graph construction, highly uneven neighbor counts, insufficient graph information extraction, and neglect of node grouping. These issues result in post-aggregation nodes with blurred features and isolated points, leading to lower accuracy in multi-class classification.

It is clear from the aforementioned publication that many researchers struggle to achieve high identification accuracy in NID, especially in multi-class classification. To address issues such as overfitting, uneven neighbor

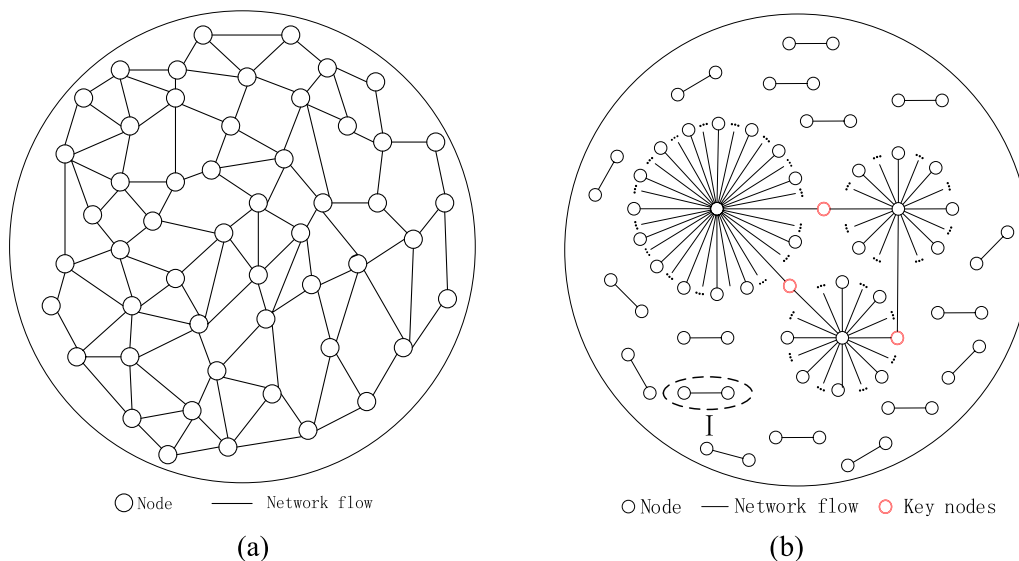


Fig. 4 **a** A graph with uniform neighbors. **b** A graph with non-uniform numbers of neighbors

distributions, and insufficient graph information extraction, this paper draws inspiration from the self-similarity of network traffic (Willinger et al. 2019) and proposes a novel graph construction method based on the principle of behavioral similarity (Yu 2014). Using this approach, edges in the created graph are given behavioral weights, which are then applied to the GAT aggregation process. Finally, the results of the experiments demonstrate a significant improvement in the accuracy of multi-class classification tasks.

Behavior similarity based GAT for NID

Problem statement and definition

In order to elucidate the issue addressed in this text, it is essential to outline the structure of the IoT in the real world. Hence, we present a diagram of the IoT model structure, depicted in Fig. 1, showcasing the collaborative interaction between cloud and edge services. Figure 1 shows that IoT devices are connected to edge servers, which provide network connectivity to these devices and forward their data packets. In recent years, with the increasing number of IoT devices, the intrusive and malicious network behaviors involving IoT devices has also risen. Therefore, the objective of this paper is to identify the behavior of the data streams received by the edge servers. Based on the above description, we can abstractly define the problem as shown in Fig. 2.

From the structure depicted in Fig. 2, we can define the problem under study in this paper. Assume an edge server provides network services to n IoT nodes, defined as $N_i(i[1, n])$, where the data flows sent by each node are defined as $v_i^k(i[1, n], k[1, \infty))$. The objective of this paper is to identify the attack type r_i^k within the data flow v_i^k .

GNN and GAT

The goal of this research is to create a model that can help edge servers identify network threats using the GAT as the training network. In recent years, GNN has emerged as one of the fastest-developing subfields of deep learning. Its ability to learn from graphs allows it to extract information that would otherwise remain hidden. Moreover, GNN possesses the adaptability to generalize to dynamic changes in network topology, rendering it particularly suitable for non-spectral graph-structured data. With the advancement of GNN, several derivatives have emerged, including GAT (Veličković et al. 2017), GCN (Kipf and Welling 2016), GraphSAGE (Hamilton et al. 2017), among others, which have garnered widespread attention from scholars. Notably, GAT stands out as a GNN that integrates attention mechanisms. This enables automatic computation of attention coefficients between nodes and subsequent aggregation of information from neighboring nodes based on these coefficients. Consequently, mining graphical data is more

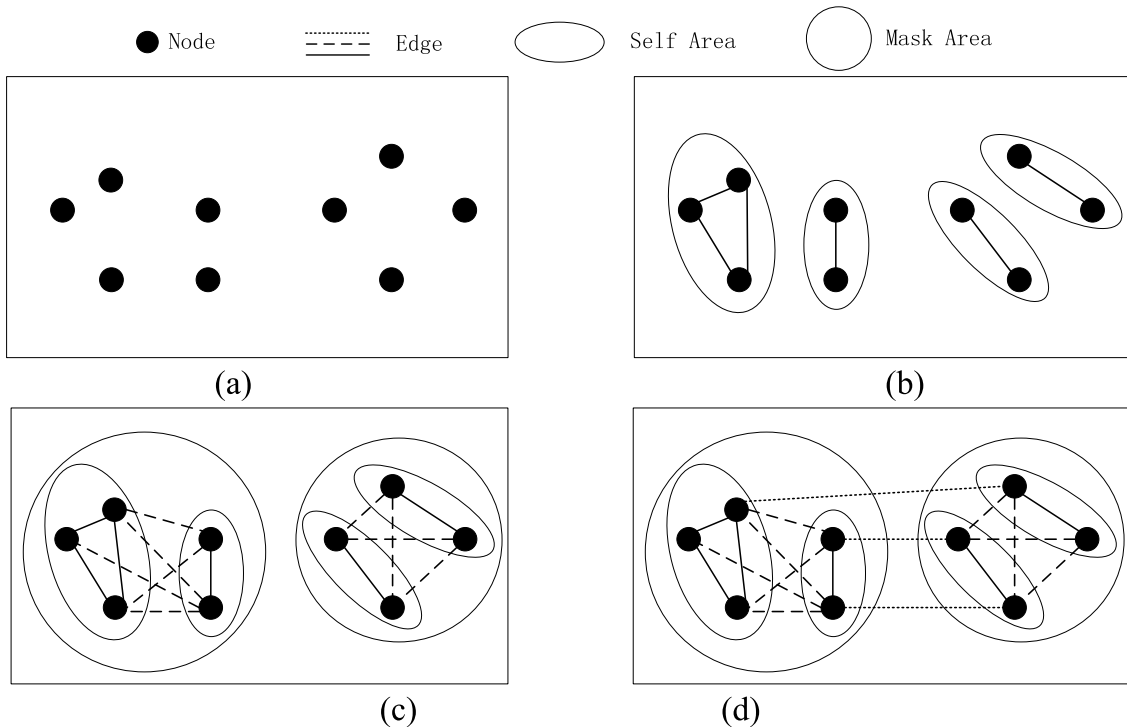


Fig. 5 Graph construction method

effective with this kind of network. This paper uses GAT to detect network intrusions.

GAT comprises two fundamental components: the computation of attention coefficients α_{ij} and the aggregating of hidden layer features $\vec{v}_i'$ of nodes.

The method for computing attention coefficients between two nodes is illustrated in the schematic on the left in Fig. 3. The original GAT paper mentions two types of neighborhoods: full neighborhoods, which include all nodes, and node neighbors, which are nodes that are directly connected to the current node. In this paper, we employ node neighbors as the neighborhood to avoid aggregating too much information from surrounding nodes. Using node neighbors can also reduce time and space complexity. The calculation equation is:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\vec{a}^T [\mathbf{W}\vec{v}_i || \mathbf{W}\vec{v}_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\vec{a}^T [\mathbf{W}\vec{v}_i || \mathbf{W}\vec{v}_k]))} \quad (1)$$

a is a single-layer feedforward neural network parameterized by a weight vector $\vec{a} \in \mathbb{R}^{2F'}$, $\mathbf{W} \in \mathbb{R}^{2F' \times F}$ is a shared linear transformation matrix, LeakyReLU is a nonlinear activation function, and $\mathcal{N}_i$ represents the neighboring nodes of node i . Finally, the equation is normalized using the *softmax* function.

The method for aggregating the hidden layer feature $\vec{v}_i'$ of node i is illustrated in the schematic on the right side of Fig. 3. The calculation equation is:

$$\vec{v}_i' = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{v}_j \right) \quad (2)$$

σ represents the nonlinear activation function. Since GAT can use a multi-head attention mechanism to aggregate hidden layer features, the number of attention heads is denoted by K .

BS based graph construction

Training a GNN model mainly consists of two steps. First, constructing a graph input for the GNN by structuring the dataset into a graph based on reasonable rules. Then, using the structure of the GNN to train the graph generated in the previous step and produce a model.

As shown in Fig. 4a, we aim to construct a graph where the number of neighbors for each node is relatively balanced. Having too many or too few neighbors can be detrimental to using GNNs for model training. When the number of neighbors is too large, it can cause the features to become blurred, leading to incorrect recognition results. A node may turn into an isolated point or edge if it does not have adequate number of neighbors. For example, the isolated edge I shown in the dashed ellipse

in Fig. 4b have no neighboring nodes, rendering it impossible for GNNs to extract any meaningful information from it. Some graph neural network-based NID methods, such as E-GraphSAGE and E-ResGAT, suffer from the issue of uneven graph structures.

IoT devices establish connections through networks, where the data traffic exchanged between these devices essentially represents their behavioral data. In this context, IoT devices can be viewed as entities similar to individuals, with network flow data analogous to human behavior. The process of identifying network flows is comparable to profiling individuals in social networks. This paper adopts a grouping approach based on the distinctive features of human behavior to categorize network traffic. This approach aids in constructing a graph with relatively uniform neighbor counts.

Because humans exhibit social attributes, their collective behavior possesses three characteristics:

- (1) An individual's multiple behaviors exhibit self-similarity.
- (2) The behaviors among different individuals within the same group exhibit some similarities.
- (3) The behaviors of individuals from different groups exhibit significant differences.

These characteristics can be correlated with the actions of IoT nodes. Data flows are considered to represent the actions of a single node if their source IP addresses are the same. They are categorized as representing the behavior of the same type of node if the subnet masks are same.

Considering the above factors, we can construct a graph of GNN for training using the following steps:

Step 1: First, we define each data flow in the dataset as a node in the graph, as illustrated in Fig. 5a. Using $\vec{v}_i$ to represent the feature vector of node i , the set of nodes in the graph is denoted as $\mathcal{V} = \{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n\}$, where n represents the number of nodes.

Step 2: According to point (1), each node $\vec{v}_i$ needs to be connected to nodes $\vec{v}_j$ that have the same source IP address, creating edges e_{ij}^s , ($i, j \in n | e_{ij}^s \in \mathcal{E}^s$). These edges and nodes form a self-similar area as shown in Fig. 5b.

Step 3: According to point (2), nodes with the same network number are grouped together using subnet masks, and edges are then constructed between nodes in the same group. The rule is to connect nodes with the same destination address and destination port number, generating edges e_{ij}^m , ($i, j \in n | e_{ij}^m \in \mathcal{E}^m$). These edges and nodes form a masked area, as shown in Fig. 5c.

Step 4: According to point (3), edges are constructed between masked regions. Node $\vec{v}_i$ searches for nodes $\vec{v}_j$ in other masked regions that have the longest network number and share the same destination IP address and destination port number as itself. These two nodes are connected to generate edge e_{ij}^o , ($i, j \in n | e_{ij}^o \in \mathcal{E}^o$), as shown in Fig. 5d

The four steps above construct the graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ used for training the GNN, where the set of nodes is denoted as $\mathcal{V}$ and the set of edges is denoted as $\mathcal{E}$. The set of edges is represented as follows:

$$\mathcal{E} = \mathcal{E}^s \cup \mathcal{E}^m \cup \mathcal{E}^o \quad (3)$$

To balance the attention mechanism of GAT using behavior similarity, behavior weights are generated for

all edges during the construction of the graph. The reasoning behind this approach will be explained later. The representation of the behavior weight matrix for edges is as follows:

$$eb_{ij} = \begin{cases} 1, & e_{ij} \in \mathcal{E}^s \\ \lambda, & e_{ij} \in \mathcal{E}^m \\ \mu, & e_{ij} \in \mathcal{E}^o \\ 0, & \text{other} \end{cases} \quad i, j \in n \quad (4)$$

Using λ and μ to represent the weight values of edges between self-similar regions and masked regions, it should be noted that the relationship between λ and μ is such that $1 < \lambda < \mu < 0$. These parameters, λ and μ , are set as hyperparameters for training the graph neural network in this paper.

The pseudo code for constructing the diagram is as follows:

Algorithm 1 Graph Construction

Input: node features $\vec{v}_i$;
Output: graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; behavior weight of edge eb_{ij} ;

```

1   $V \leftarrow \vec{v}$ 
2  for  $i = 1, \dots, n$  do      //  $n$  is the number of nodes
3      for  $j = i, \dots, n$  do
4          if  $\vec{v}_i$  and  $\vec{v}_j$  have the same source IP address
5               $\mathcal{E} = \mathcal{E} \cup e_{ij}^s$ 
6               $eb_{ij} = 1$ 
7              continue
8          end
9          if  $\vec{v}_i$  and  $\vec{v}_j$  are in the network number and have the same destination IP
              address and destination port
10              $\mathcal{E} = \mathcal{E} \cup e_{ij}^m$ 
11              $eb_{ij} = \lambda$ 
12             continue
13         end
14         if  $\vec{v}_i$  and  $\vec{v}_j$  have the longest prefix in the subnet mask and have the
              same destination port, destination IP address
15              $\mathcal{E} = \mathcal{E} \cup e_{ij}^o$ 
16              $eb_{ij} = \mu$ 
17             continue
18         end
19     end
20 end
21  $\mathcal{G} \leftarrow \mathcal{V}, \mathcal{E}$ 

```

From the loop structure in the pseudo code, it can be observed that the algorithm's time complexity is $O\left(\frac{n^2}{2}\right)$, where n depends on the number of data flow in the dataset.

BS based GAT

According to the GAT algorithm, all attention coefficients which are comparable to the similarity values between a node and its neighboring nodes are automatically trained via a neural network. Part 3.3 of this study describes a graph whose information is incompletely captured when neighboring nodes are aggregated using automatically learnt attention coefficients.

Attention coefficients, which represent the values assigned to connections between nodes, should reflect behavioral similarities. When aggregating features from the hidden layers of nodes, the process should be

influenced by the weighting of edge behaviors. Hence, we introduce behavior weighting into the calculation of attention coefficients:

$$\alpha_{ij} = \frac{eb_{ij}(\exp(\text{LeakyReLU}(\vec{a}^T[W\vec{v}_i||W\vec{v}_k])))}{\sum_{k \in \mathcal{N}_i} eb_{ik}(\exp(\text{LeakyReLU}(\vec{a}^T[W\vec{v}_i||W\vec{v}_k])))} \quad (5)$$

Behavioral similarity weight is incorporated into the aggregation process of node hidden layer features:

$$\vec{v}_i' = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} eb_{ij} \alpha_{ij}^k W^k \vec{v}_j \right) \quad (6)$$

The constraint method illustrated by Eq. (5) in this paper is adopted primarily because behavior weights can influence all neighboring nodes when calculating attention coefficients.

The pseudocode for BS-GAT is as follows:

Algorithm 2 BS-GAT minibatch aggregation

Input: Graph $\mathcal{G}(V, E)$;
node features $\vec{v}_i$;
layer L ; number of heads K ;
weight matrices $W^k, \forall k \in \{1, \dots, K\}$;
non-linearity σ ;
neighborhood for node i $\mathcal{N}_i$;
behavior weight of edge eb_{ij} ;

Output: node features $\vec{v}_i^l$;

```

1   $\vec{v}_i^0 \leftarrow \vec{v}_i$ 
2  for  $l = 1, \dots, L$  do
3    for  $k = 1, \dots, K$  do
4       $\alpha_{ij}^k = \frac{eb_{ij}(\exp(\text{LeakyReLU}(\vec{a}^T[W^k \vec{v}_i || W^k \vec{v}_j])))}{\sum_{s \in \mathcal{N}_i} eb_{is}(\exp(\text{LeakyReLU}(\vec{a}^T[W^k \vec{v}_i || W^k \vec{v}_s])))}$ 
5    end
6     $\vec{v}_i^l = \sigma(\frac{1}{K} \sum_{k=1}^K \sum_{s \in \mathcal{N}_i} \alpha_{ij}^k W^k \vec{v}_s)$ 
7  end

```

Table 1 The information of data sets

Dataset	Release year	No. Classes	No. features	No. data	Benign ratio
BoT	2020	5	12	3,668,522	0.2 to 9.8
BoT-v2	2021	5	43	37,763,497	0.0 to 10.0
ToN	2020	10	12	22,339,021	2.0 to 8.0
ToN-v2	2021	10	43	16,940,496	3.6 to 6.4

From the loop structure in the pseudo code, it can be observed that the algorithm's time complexity is $O\left(\frac{n^2}{2}\right)$, where n depends on the number of data flow in the dataset.

Experimental evaluation

Data sets

This paper will use the datasets NF-BoT-IoT-v2 (BoT-v2) and NF-ToN-IoT-v2 (ToN-v2) to validate method proposed. These two datasets were published in the work by Sarhan et al. (2022), which primarily focuses on improving the datasets NF-BoT-IoT and NF-ToN-IoT

introduced in the paper (Sarhan, et al. 2020). The main information about these datasets is presented in Table 1. The fields of NID make extensive use of these two datasets, as noted in studies (Zeng et al. 2022; Nuaimi et al. 2023; Anjhpsp et al. 2023; Magán-Carrión et al. 2022). These datasets have been used in hundreds of publications to validate various NID techniques.

The primary contribution of these datasets is the use of network analysis tools to reformat the real datasets BoT-IoT (Koroniotis et al. 2019) and ToN-IoT (Moustafa 2021). The most significant change in the v2 version of these datasets is the expansion of the number of feature fields, which greatly benefits classification and

Table 2 The information of multi-class

Dataset	Classes (names and %)									
BoT-v2	Benign	Reconnaissance	DDos	Dos	Theft					
	0.36	6.94	48.54	44.15	0.0064					
ToN-v2	Benign	Reconnaissance	DDos	Dos	Backdoor	Injection	MITM	Password	Scanning	XSS
	36.01	0.02	11.96	4.2	0.0099	4.04	0.0046	6.8	22.32	14.49

Table 3 Real example of the extended dataset BoT-v2

FI									CLASS	
IPV4_SRC_ADDR	L4_SRC_PORT	...	L7_PROTO	IN_BYTES	IN_PKTS	...	DNS_TTL_ANSWER	FCMRC	LABEL	ATTACK
192.168.100.148	59,826	...	188	56	2	...	0	0	1	DDos
192.168.100.150	61,662	...	7	280	2	...	0	0	1	DoS
192.168.100.147	47,080	...	1	13	1	...	0	331	1	reconnaissance
192.168.100.3	43,362	...	0	60	1	...	0	0	1	Theft
192.168.100.3	80	...	7	44	1	...	0	0	0	Benign

Table 4 Real example of the extended dataset ToN-v2

FI									CLASS	
IPV4_SRC_ADDR	L4_SRC_PORT	...	L7_PROTO	IN_BYTES	IN_PKTS	...	DNS_TTL_ANSWER	FCMRC	LABEL	ATTACK
192.168.1.193	49,228	...	0	1600	40	...	86,400	0	0	Benign
192.168.1.31	59,779	...	0	50	1	...	0	0	1	ddos
192.168.1.31	37,514	...	0	70	1	...	23	0	1	dos
192.168.1.31	34,165	...	0	54	1	...	0	0	1	injection
192.168.1.31	60,540	...	0	50	1	...	0	0	1	mitm
192.168.1.31	60,918	...	7	84	2	...	0	0	1	password
192.168.1.33	4444	...	0	28,296	133	...	0	0	1	reconnaissance
192.168.1.32	41,244	...	3	44	1	...	0	0	1	scanning
192.168.1.32	46,187	...	0	70	1	...	57	0	1	xss
192.168.1.79	60,766	...	0	63	1	...	0	0	1	backdoor

identification. Multi-class information is summarized in Table 2, providing the information of categories and the ratio of each category to the total dataset.

In Tables 3 and 4, sample data are included in this research to give readers a better grasp of the datasets. This makes it easier for readers to understand the issue that this work seeks to solve.

Due to the large volumes of data in the BoT-v2 and ToN-v2 datasets, with 37,763,497 and 16,940,496 instances respectively, training on the entire dataset would be highly time-consuming. Therefore, this paper adopts a partial data approach for experimentation. Each experiment selects data that encompasses all categories in the dataset and is repeated multiple times. The average of all results from these repetitions is then taken as the final result. Additionally, the dataset used for each experiment is divided into three parts: 50% of the dataset is utilized for training, 20% for validation, and the remaining 30% for testing the model's accuracy. This approach not only increases efficiency but also demonstrates the stability of the proposed algorithm.

Experimental setting

(A) Evaluation Metrics

This paper's experiment is segmented into two sections: assessing node neighbor uniformity and evaluating classification accuracy. Consequently, the experiment will discuss evaluation metrics specific to each of these sections.

(1) Average degree of the number of neighbors of a node

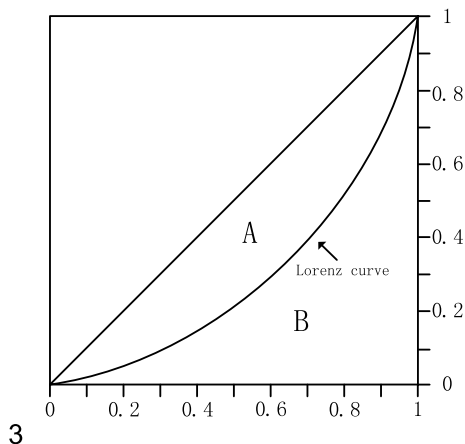


Fig. 6 Gini coefficient diagram

This paper utilizes the Gini coefficient (Dorfman 1979) to assess the average degree of a node's neighbors. Equation (7) and Fig. 6 below outline the calculation method and schematic diagram, respectively. The Gini coefficient primarily measures the income inequality among residents, with values ranging from 0 to 1. A higher Gini coefficient indicates greater income disparity, while a lower value suggests less disparity and more uniform income distribution among all residents. In this paper, the Gini coefficient is used to represent the disparity in the number of neighbors a node acquires after graph generation. A higher Gini coefficient indicates a more uneven distribution of neighbors among node, whereas a lower value indicates a more even distribution. The application of the Gini coefficient in network intrusion detection is also quite extensive (Li et al. 2018; Disha and Waheed 2022; Zhang et al. 2021b). Interested readers can refer to the provided literature for more detailed information.

$$G = \frac{S_A}{S_A + S_B} \quad (7)$$

(2) Classification accuracy

To assess the identification accuracy of the NID method proposed in this paper, the evaluation metrics from Table 5 are employed to compare it with previously proposed algorithms (KNN, SVM, Random Forest, Extra Trees, E-GraphSAG, E-ResGAT, GAT), Extra Trees is the classification method used by the dataset provider. The baseline methods compared in this paper have been extensively cited in these literature (Chang and Paula 2021; Khan et al. 2023; Aljuhani et al. 2022; Ma et al. 2023; Altaf et al. 2023; Shuaiyi et al. 2023) on network intrusion detection and are frequently used as benchmarks for comparison. In addition, these metrics are considered crucial and classical in the field of classification recognition, widely utilized across numerous articles.

Table 5 Evaluation metrics adopted

Metric	Definition
Recall	$\frac{TP}{TP+FN}$
Precision	$\frac{TP}{TP+FP}$
F1-Score	$2 \times \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$
Accuracy	$\frac{TP+TN}{TP+FP+FN+TN}$
False Negative Rate(FNR)	$\frac{FN}{FN+TP}$
False Positive Rate(FPR)	$\frac{FP}{TN+FP}$

Table 6 Experimental hyperparameter settings

Name of parameter	BS-GAT	E-GraphSAG	E-ResGAT	GAT
Learning rate	0.002	0.001	0.003	0.002
Dropout	0.2	0.2	0.2	0.2
Hop	1	2	2	1
Hidden	128	128	128	128
Layers	2	2	2	2
Minibatches	500	500	500	500
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy	Cross-Entropy
Optimizer	Adam	Adam	Adam	Adam
Epoch	500	500	500	500
Nonlinear function	eLU	ReLU	ReLU	eLU
nb_heads	3	–	6	3
λ	0.85	–	–	–
μ	0.7	–	–	–

Table 7 Statistical results of the Gini coefficient

	Gini coefficient	
	BoT-v2	ToN-v2
BS-GAT	0.3529	0.4203
E-GraphSAGE	0.8845	0.9216
E-ResGAT	0.8903	0.9176

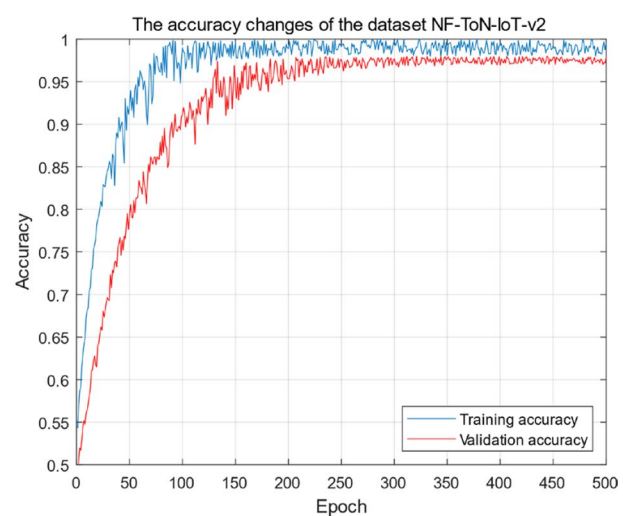
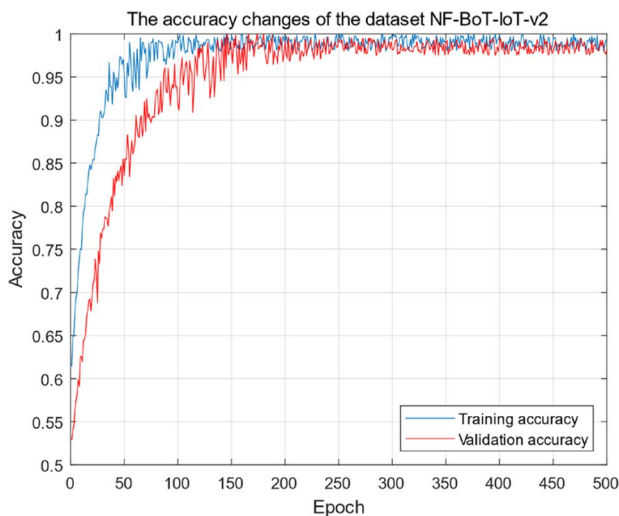
(B) Parameter setting

In the experiment, the hyperparameters used for training each method are presented in Table 6. Except for BS-GAT, which has specific hyperparameter values set, the other methods utilize the hyperparameter

values provided in their original papers. After training the model with these hyperparameters, feature vectors for each node can be obtained from the final layer of the GNN after information extraction. These feature vectors are then input into a fully connected layer for classification and identification. It should be noted that in this paper, during experimentation, λ and μ were set to 0.85 and 0.7 respectively. The main reason for this choice is to fine-tune the behavior weights for GAT training. Setting them too small would have a significant impact on training.

Results

(A) Gini coefficient.

**Fig. 7** The accuracy changes during binary classification training for the two datasets

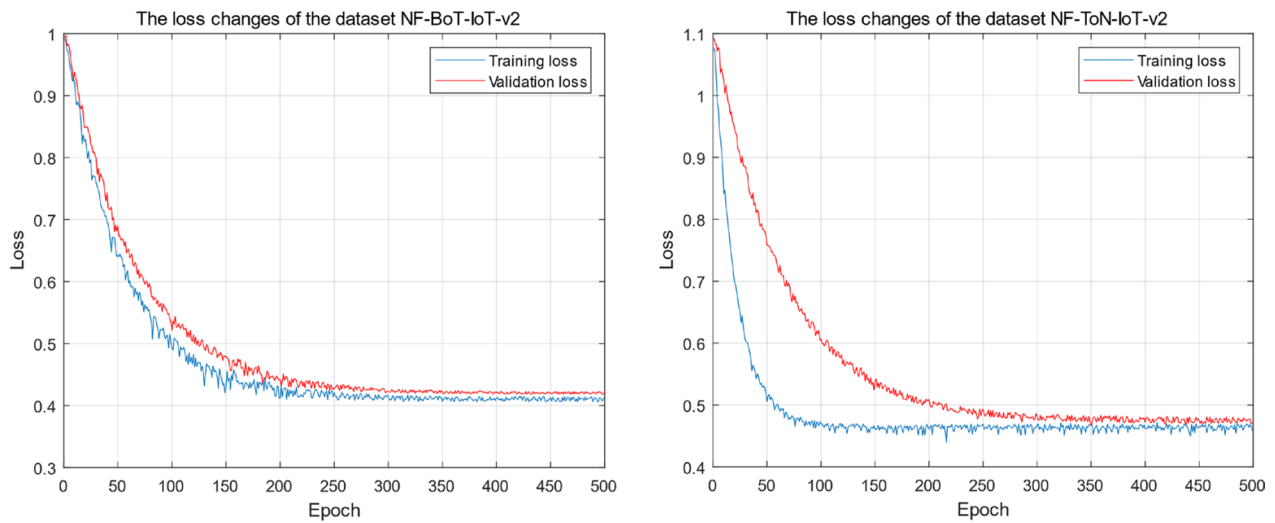


Fig. 8 The loss changes during binary classification training for the two datasets

The proposed method for creating graphs has a lower Gini coefficient than the other two approaches, as shown in the results in Table 7. This indicates that the graphs constructed by the proposed method are more uniform, providing a clear advantage when using GNNs for recognition and classification.

(B) Classification results

The dataset's final entries comprise two label fields, which can be used for binary classification and multi-class classification predictions. We primarily collected statistics on the binary classification and multi-class classification accuracy of the methods described in this

paper. Additionally, we plotted the changes in accuracy and loss during the training process to illustrate the progression of these metrics.

(1) Binary classification results

Figures 7 and 8 illustrate the accuracy and loss variation during the binary classification training process for two datasets. Figure 7 shows that the training accuracy quickly reaches above 97%, with values of 97% and 98% respectively. This indicates that the method proposed in this paper performs well on binary classification tasks. Figure 8 demonstrates that the training loss also rapidly decreases to within the ideal range, with training

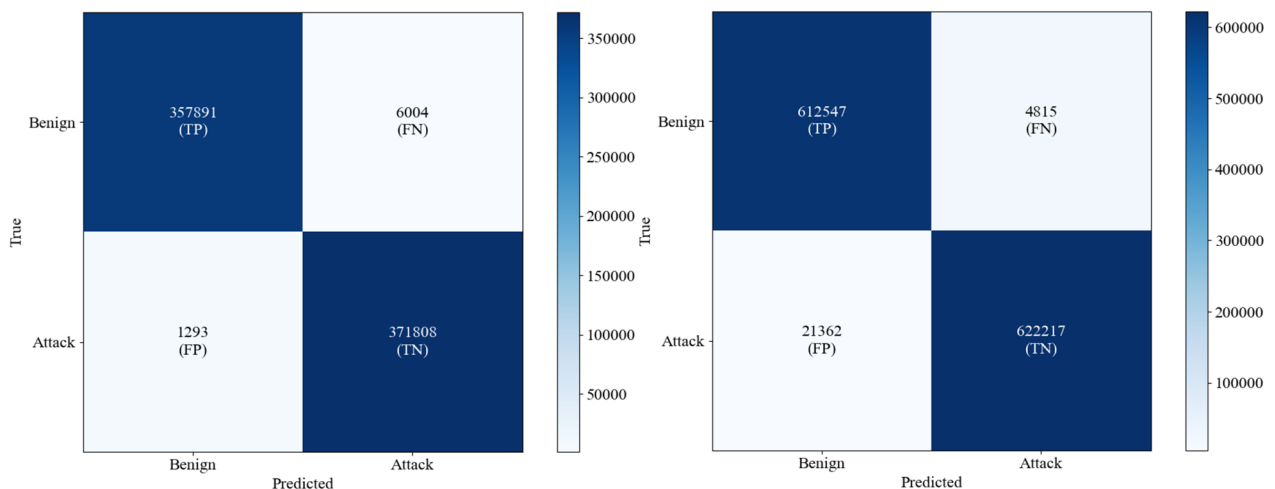


Fig. 9 Confusion matrices for the binary classification results of the datasets. Left: Dataset BoT-v2. Right: Dataset ToN-v2

Table 8 Results of FPR and FNR

Dataset	Algorithm	FPR	FNR
BoT-v2	KNN	0.1495	0.3857
	SVM	0.0976	0.2604
	Random Forest	0.1438	0.2331
	Extra Trees	0.0715	0.0376
	GAT	0.0908	0.1087
	E-GraphSAG	0.0633	0.0291
	E-ResGAT	0.0292	0.0225
	BS-GAT	0.0152	0.007
ToN-v2	KNN	0.1404	0.2571
	SVM	0.1422	0.1482
	Random Forest	0.1509	0.1968
	Extra Trees	0.0421	0.0352
	GAT	0.0688	0.0783
	E-GraphSAG	0.053	0.0167
	E-ResGAT	0.0754	0.0375
	BS-GAT	0.0033	0.0127

Table 9 Results of each algorithm on binary classification

Dataset	Algorithm	Recall	Precision	F1-Score	Accuracy
BoT-v2	KNN	0.7425	0.7017	0.7319	0.7859
	SVM	0.8512	0.8727	0.8904	0.8299
	Random Forest	0.8029	0.8191	0.7954	0.7923
	Extra Trees	0.9451	0.8715	0.9228	0.923
	GAT	0.9216	0.9113	0.8769	0.9213
	E-GraphSAG	0.9831	0.9461	0.9642	0.9635
	E-ResGAT	0.9621	0.92	0.9406	0.9392
	BS-GAT	0.9835	0.9964	0.9899	0.99
ToN-v2	KNN	0.6142	0.68	0.6276	0.7012
	SVM	0.7396	0.7921	0.7925	0.8206
	Random Forest	0.7667	0.7126	0.7888	0.7921
	Extra Trees	0.9412	0.9278	0.9344	0.9014
	GAT	0.8912	0.8617	0.8214	0.8917
	E-GraphSAG	0.9705	0.9339	0.9518	0.9509
	E-ResGAT	0.977	0.9696	0.9735	0.9734
	BS-GAT	0.9922	0.9663	0.9790	0.9788

losses for both datasets being below 0.5, at 0.42 and 0.47 respectively.

The clear stability in both accuracy and loss fluctuations post-200 epochs suggests robustness in the proposed method. This observation underscores the method's reliability, affirming its efficacy in binary classification for intrusion detection.

To further analyze the effectiveness of the proposed method, the confusion matrices for binary classification on two datasets, as shown in Fig. 9, are provided. It can be seen that the dataset BoT-v2 has false positives (FP) and false negatives (FN) of 1,293 and 6,004, respectively, accounting for 0.34% and 1.64% of the total samples. The dataset ToN-v2 has FP and FN values of 21,362 and 4,815, respectively, accounting for 3.32% and 0.78% of the total samples. This demonstrates that the proposed method has very low FPR and FNR in data stream detection.

FPR and FNR are crucial for network intrusion detection. A false negative means identifying an attack flow as benign, while a false positive means identifying a benign flow as an attack. If these two metrics are high, it is very detrimental to network intrusion detection. Therefore, this paper compares the FPR and FNR across different methods, as shown in Table 8. The results indicate that the proposed method has lower FPR and FNR than other methods.

Since only a portion of the dataset was used for each experiment, Table 9 displays the average results of various metrics for seven methods in binary classification. Among them, both E-GraphSAG and E-ResGAT exhibit recall rates exceeding 96% on the two datasets, consistent with their reported results. However, the algorithm proposed in this paper still outperforms the other six algorithms on both datasets. Additionally, the proposed algorithm demonstrates excellent performance in precision as well. On the ToN-v2 dataset, its performance is only 0.33% lower than E-ResGAT but surpasses GAT and E-GraphSAG by 10.46% and 3.24%, respectively. Furthermore, the proposed algorithm outperforms the other six algorithms in F1-Score, indicating its robustness.

(2) Multiclass classification results

The accuracy and loss changes during training for two datasets on multiclass classification are shown in Figs. 10 and 11. As seen in Fig. 10, training accuracy reaches over 90% in both cases, specifically 92% and 93%, respectively. This indicates a significant improvement of the proposed method in multiclass classification. Additionally, Fig. 11 shows training loss rapidly decreases to an ideal range. The training loss for both datasets is below 1.3, specifically 0.8 and 1.3, respectively.

Similar to the binary classification, this paper also presents the confusion matrix for multi-class classification results, as shown in Fig. 12. The diagonal elements of

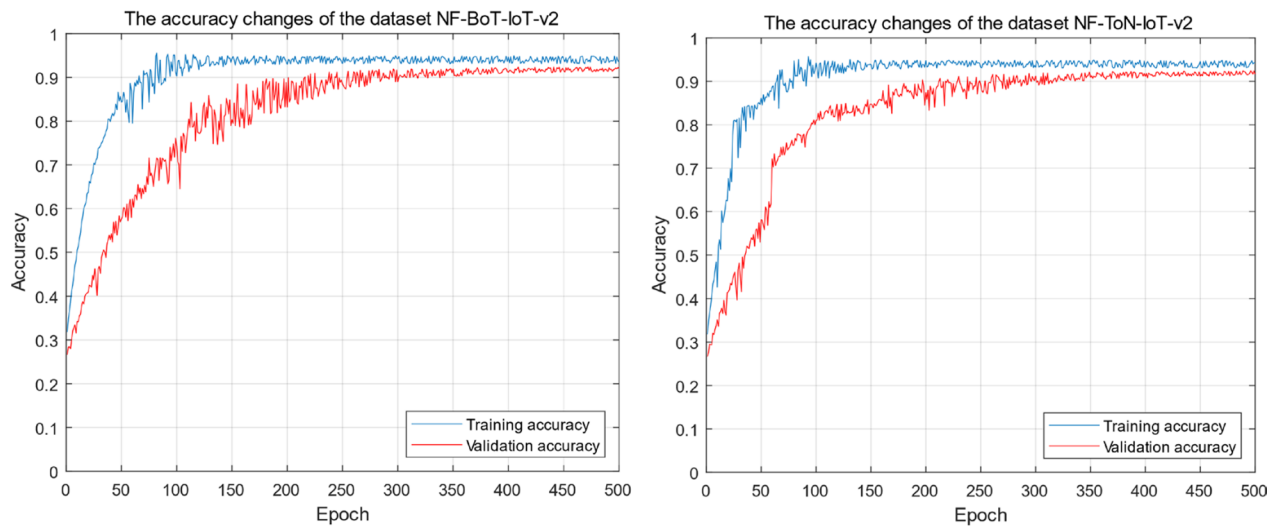


Fig. 10 The accuracy changes during multi-class classification training for the two datasets

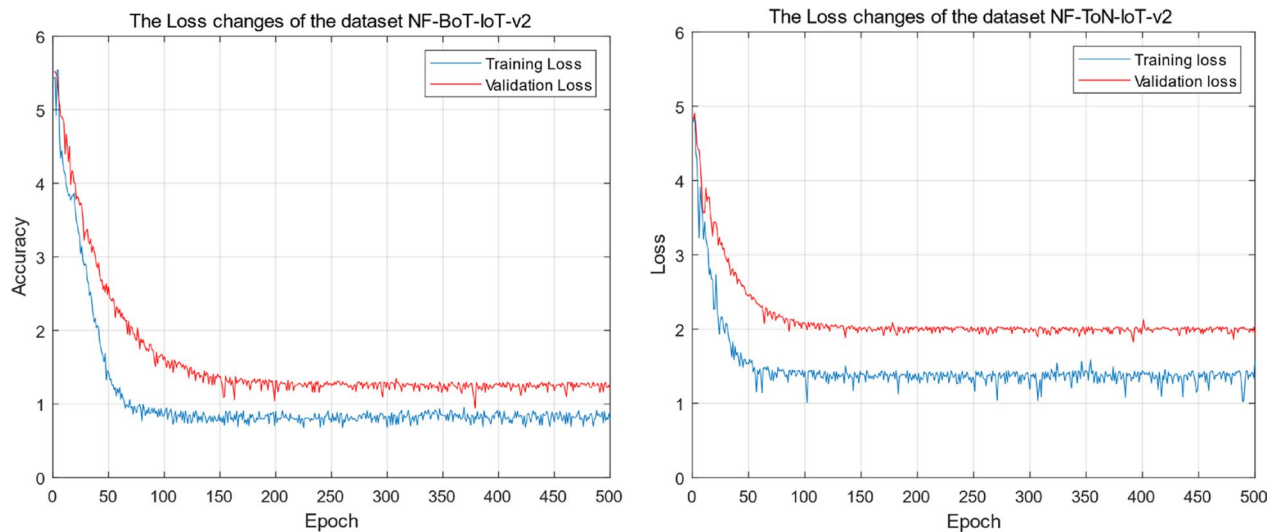


Fig. 11 The loss changes during multi-class classification training for the two datasets

the confusion matrix represent the number of correctly predicted samples by the model. As can be seen from the figure, most of the diagonal elements have high values, indicating that the model performs well in predicting the majority of classes. Notably, it shows high recognition rates for categories such as Benign, Backdoor, MITM, Password, and XSS.

For the multiclass problem, this paper also adopts the method of multiple experiments followed by averaging to conduct experiments. Then, it statistically records the accuracy of each classification as well as the overall performance indicators. The results are shown in Tables 10 and 11. Table 10 shows that the recognition accuracy of each category on the two datasets is relatively high

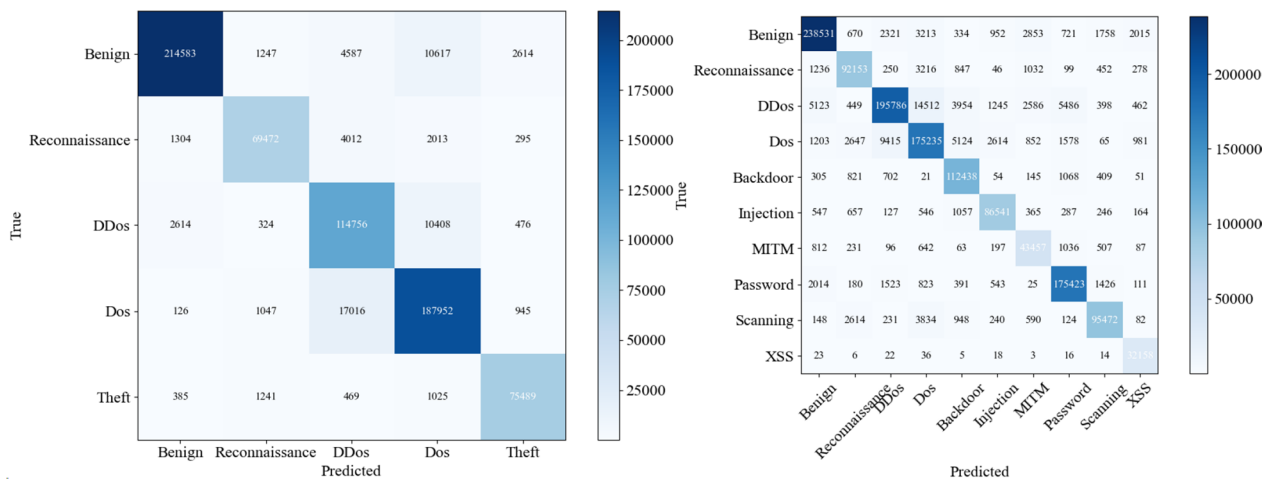


Fig. 12 Confusion matrices for the multi-class classification results of the datasets. Left: Dataset BoT-v2. Right: Dataset ToN-v2

Table 10 Recall results per class on the two datasets

Dataset	Algorithm	Per class Recall									
BoT-v2		Benign	Reconnaissance	DDos	Dos	Theft					
	KNN	0.4419	0.3754	0.6025	0.6136	0.4526					
	SVM	0.704	0.6148	0.8012	0.8149	0.7225					
	Random Forest	0.8205	0.8415	0.7451	0.5748	0.8148					
	Extra Trees	0.8414	0.7586	0.5892	0.8617	0.9076					
	GAT	0.8552	0.6715	0.6216	0.7069	0.7149					
	E-GraphSAG	0.8256	0.8012	0.7646	0.7951	0.8494					
	E-ResGAT	0.7662	0.8027	0.6993	0.7119	0.8226					
	BS-GAT	0.9184	0.9011	0.8925	0.9076	0.9603					
ToN-v2		Benign	Reconnaissance	DDos	Dos	Backdoor	Injection	MITM	Password	Scanning	XSS
	KNN	0.5136	0.445	0.3652	0.4795	0.6584	0.5456	0.6458	0.7016	0.6475	0.6852
	SVM	0.6514	0.6268	0.5243	0.5813	0.7213	0.6912	0.6862	0.802	0.7958	0.8056
	Random Forest	0.7054	0.7258	0.6469	0.6549	0.7608	0.8647	0.6058	0.6926	0.6746	0.8106
	Extra Trees	0.9015	0.8426	0.6722	0.8214	0.9245	0.8713	0.7596	0.7169	0.8512	0.8612
	GAT	0.7512	0.7024	0.7325	0.6392	0.7104	0.8024	0.552	0.7425	0.8984	0.8156
	E-GraphSAG	0.8943	0.9634	0.7947	0.744	0.5811	0.6328	0.7651	0.8023	0.8042	0.825
	E-ResGAT	0.6996	0.8457	0.7228	0.606	0.8226	0.7825	0.8044	0.8419	0.8831	0.7639
	BS-GAT	0.9414	0.9251	0.8512	0.8774	0.9692	0.9559	0.9221	0.9614	0.9155	0.9956

accuracy rates compared to other algorithms, indicating a significant improvement. It is important to note that while some classifications achieve an accuracy of up to 99.56%, DDos only reaches 85.15%. A major reason for this result is the high similarity between certain types of attacks, such as Dos and DDos. As seen in the confusion matrix, more than 6.31% of DDos attacks were misclassified as Dos, and over 4.7% of Dos attacks were

misclassified as DDos. This paper suggests that distinguishing between these similar attacks is key to improving the recognition accuracy, and this will be the focus of our next steps. In the overall performance indicators shown in Table 11, there is at least an 8% improvement in both the weighted recall and weighted f1-score metrics, with some performance improvements exceeding 40%. This indicates that the algorithm proposed in this paper

Table 11 Performance results for each algorithm over multiple classifications

Dataset	Algorithm	Weighted Recall	Weighted F1-Score
BoT-v2	KNN	0.5232	0.5761
	SVM	0.6923	0.711
	Random Forest	0.7417	0.7727
	Extra Trees	0.7524	0.8014
	GAT	0.7014	0.6593
	E-GraphSAG	0.8012	0.8407
	E-ResGAT	0.7263	0.7331
	BS-GAT	0.9018	0.9247
ToN-v2	KNN	0.5105	0.5448
	SVM	0.62	0.6552
	Random Forest	0.6911	0.6244
	Extra Trees	0.8017	0.8317
	GAT	0.7186	0.7405
	E-GraphSAG	0.8207	0.8026
	E-ResGAT	0.7159	0.7566
	BS-GAT	0.9141	0.9022

has a very significant effect on multiclass tasks and possesses good practical value.

This study demonstrates the effectiveness of the proposed approach in detecting network intrusions through binary and multi-classification studies.

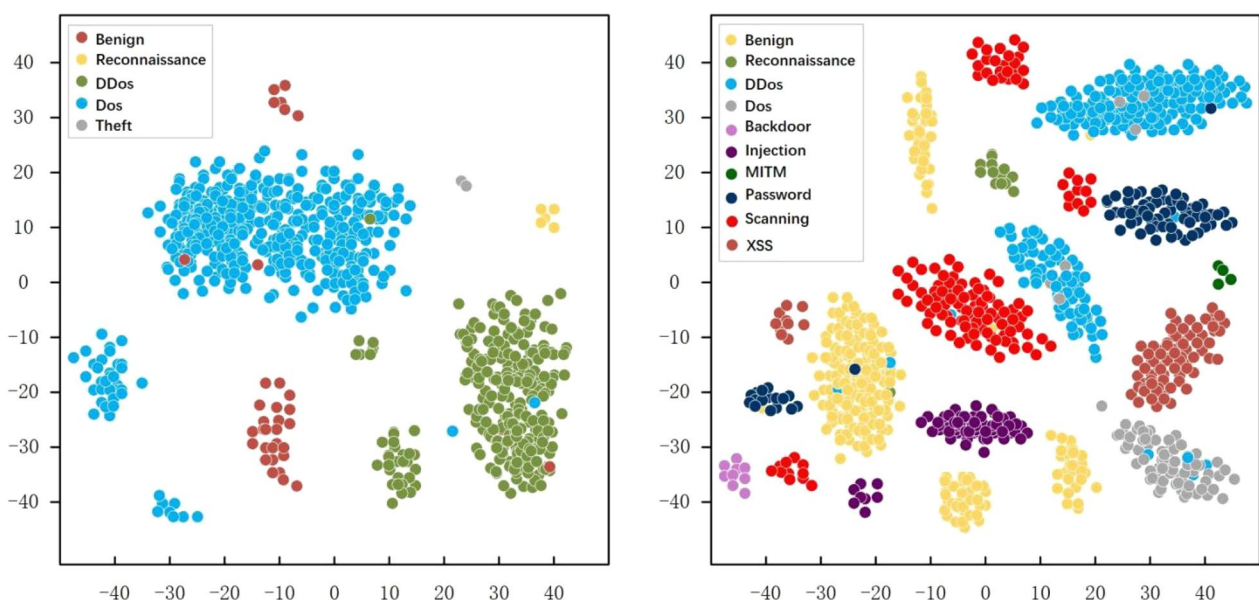
The t-SNE method is a technique used to transform multi-dimensional features into a visually interpretable form. Therefore, to more intuitively analyze the clustering effect of the proposed method, we used t-SNE to display

the node features from the final layer of our method. The results are shown in Fig. 13.

Conclusions

To address the issue of NID, this paper proposes an advanced graph neural network-based method that introduces the concept of behavior similarity as the logical rule for graph construction and as a training constraint for the GNN. The core approach involves grouping data flows as nodes in the graph, constructing edges based on behavior rules, and incorporating the behavior weight matrix into the training of the GAT. This method addresses issues such as uneven neighbor distribution among nodes, insufficient information mining, and overfitting in GNNs. The final experimental results show that the proposed method effectively improves classification accuracy, particularly in multi-class classification.

Despite a significant improvement in the precision of multi-class classification, the recognition performance for certain attacks in multi-class scenarios remains unsatisfactory, such as Dos in the ToN-v2 dataset. A possible cause of this problem is insufficient integrity of the data set. In fact, an important aspect of identifying a network stream data is to obtain the state of the sender of the stream data. For example, during Dos attacks, the network parameters of the sender will be very active. Therefore, in the future it will be necessary to improve the existing dataset and study it to make it develop better solutions to multi-classification problems.

**Fig. 13** The hidden layer features of datasets BoT-v2 (Left) and ToN-v2 (Right) visualized using the t-SNE method

Acknowledgements

This work was supported by the Special Project for Key R&D and the Promotion of Science, Technology Department of Henan Province (222102210272, 232102210071 and 232102211009), the Major Science and Technology Projects of Henan Province, and the Major Public Welfare Projects of Henan Province (201300210400), Kaifeng Science and Technology Development Plan (2201010).

Author contributions

The contributions of all authors in various aspects are as shown above. These authors all played indispensable roles in the development of the article. Investigation and research, Y.W. and Z.H.; Concept introduction Y.W. and Y.D.; Methodology, Y.W. and Y.D.; Data Organization, J.L.; Formatted Analysis, Z.H.; Resources, Y.W. and X.H.; Regulation, Y.D.; Project Management, J.L.; Software, Y.W. and Z.H.; Verification, Y.W., Z.H. and D.Y.; Original Draft, Y.W.; Writing and Editing, W.Z., Z.H. and X.H.

Funding

Not applicable.

Availability of data and materials

The datasets generated and analysed during the current study are available in the Machine Learning-Based NIDS Datasets repository, this link is https://staff.itee.uq.edu.au/marius/NIDS_datasets/. The other datasets used and analysed during the current study are available from the corresponding author on reasonable request.

Declaration

Competing interests

The authors declare that they have no competing interests.

Received: 29 May 2024 Accepted: 26 September 2024

Published online: 24 April 2025

References

- Ahmad I, UI Haq QE, Imran M et al (2022) An efficient network intrusion detection and classification system. *Mathematics* 10(3):530. <https://doi.org/10.3390/math10030530>
- Aljuhani A, Kumar P, Kumar R et al (2022) Fog intelligence for secure smart villages: architecture and future challenges. *IEEE Consum Electron Mag* 12(5):12–21. <https://doi.org/10.1109/MCE.2022.3193268>
- Altaf T, Wang X, Ni W et al (2023) A new concatenated multigraph neural network for iot intrusion detection. *Internet of Things* 22:100818. <https://doi.org/10.1016/j.iot.2023.100818>
- Anjhsap S et al (2023) On-device context-aware misuse detection framework for heterogeneous IoT edge. *Appl Intell* 53:14792–14818. <https://doi.org/10.1007/s10489-022-04039-5>
- Bhuyan MH, Bhattacharyya DK, Kalita JK (2011) Surveying port scans and their detection methodologies. *Comput J* 54(10):1565–1581. <https://doi.org/10.1093/comjnl/bxr035>
- Chang L and Paula B (2021) Graph-based Solutions with Residuals for Intrusion Detection: the Modified E-GraphSAGE and E-ResGAT Algorithms. *arXiv preprint arXiv:2111.13597*. <https://doi.org/10.48550/arXiv.2111.13597>
- Churcher A, Ullah R, Ahmad J, Rehman S, Masood F, Gogate M, Alqahtani F, Nour B, Buchanan WJ (2021) An experimental analysis of attack classification using machine learning in IoT networks. *Sensors* 21:2. <https://doi.org/10.3390/s21020446>
- Disha RA, Waheed S (2022) Performance analysis of machine learning models for intrusion detection system using Gini Impurity-based Weighted Random Forest (GIWRF) feature selection technique. *Cybersecurity* 5(1):1. <https://doi.org/10.1186/s42400-021-00103-8>
- Dorfman R (1979) A formula for the Gini coefficient. *Rev Econ Stat* 5:146–149. <https://doi.org/10.2307/1924845>
- Ghasempour (2019) Internet of things in smart grid: Architecture, applications, services, key technologies, and challenges. *Inventions* 4(1): 22. <https://doi.org/10.3390/inventions4010022>
- Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. In: *Proceedings of the 31st international conference on neural information processing systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., pp 1025–1035, ISBN: 9781510860964 Available Link: https://proceedings.neurips.cc/paper_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7e9bea9-Abstract.html
- He H, Sun X, He H, Zhao G, He L, Ren J (2019) A novel multimodal-sequential approach based on multiview features for network intrusion detection. *IEEE Access* 7:183207–183221
- Islam R, Refat RUD, Yerram SM et al (2020) Graph-based intrusion detection system for controller area networks. *IEEE Trans Intell Transp Syst* 23(3):1727–1736. <https://doi.org/10.1109/TITS.2020.3025685>
- Jiang W (2021) Graph-based deep learning for communication networks: a survey. *Comput Commun* 185:40–54. <https://doi.org/10.1016/j.comcom.2021.12.015>
- Jiang W, Jiayun L (2022) Graph neural network for traffic forecasting: a survey. *Expert Syst Appl*. <https://doi.org/10.1016/j.eswa.2022.117921>
- Kambourakis G, Moschos T, Geneiatakis D, Gritzalis S (2007) Detecting dns amplification attacks. In: *International workshop on critical information infrastructures security*, Springer, pp 185–196. https://doi.org/10.1007/978-3-540-89173-4_16
- Khan NW, Alshehri MS, Khan MA et al (2023) A hybrid deep learning-based intrusion detection system for IoT networks. *Math Biosci Eng* 20(8):13491–13520
- Kipf TN, Welling M (2016) Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, <https://doi.org/10.48550/arXiv.1609.02907>
- Koroniotis N, Moustafa N, Sitnikova E, Turnbull B (2019) Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Futur Gener Comput Syst* 100:779–796. <https://doi.org/10.1016/j.future.2019.05.041>
- Lawal MA, Shaikh RA, Hassan SR (2020) An anomaly mitigation framework for IoT using fog computing. *Electronics* 9:10. <https://doi.org/10.3390/electronics9101565>
- Li L, Yu Y, Bai S et al (2018) Towards effective network intrusion detection: a hybrid model integrating gini index and GBDT with PSO. *J Sens* 2018(1):1578314. <https://doi.org/10.1155/2018/1578314>
- Lo WW et al. (2022) E-GraphSAGE: a graph neural network based intrusion detection system for IoT. In: *NOMS 2022–2022 IEEE/IFIP network operations and management symposium*. IEEE. <https://doi.org/10.1109/NOMS54207.2022.9789878>
- Ma Z, Liu L, Meng W et al (2023) ADCL: toward an adaptive network intrusion detection system using collaborative learning in IoT networks. *IEEE Internet Things J* 10(14):12521–12536. <https://doi.org/10.1109/JIOT.2023.3248259>
- Magán-Carrión R, Urda D, Diaz-Cano I et al (2022) Improving the reliability of network intrusion detection systems through dataset integration. *IEEE Trans Emerg Top Comput* 10(4):1717–1732. <https://doi.org/10.1109/TETC.2022.3178283>
- Moustafa N (2021) A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustain Cities Soc*. <https://doi.org/10.1016/j.scs.2021.102994>
- Nuaimi M, Fourati LC, Hamed BB (2023) Intelligent approaches toward intrusion detection systems for Industrial Internet of Things: a systematic comprehensive review. *J Netw Comput Appl*. <https://doi.org/10.1016/j.jnca.2023.103637>
- Qingsai X, Jian L, Quiyun W, Zhengwei J, Xuren W, Yepeng Y (2020) Towards network anomaly detection using graph embedding. In: *International Conference on Computational Science*, pp 156–169. Springer, https://doi.org/10.1007/978-3-030-50423-6_12
- Sarhan M, Layeghy S, Portmann M (2022) Towards a standard feature set for network intrusion detection system datasets. *Mob Netw Appl* 27(1):357–370. <https://doi.org/10.1007/s11036-021-01843-0>
- Sarhan M, Layeghy S, Moustafa N, Portmann M (2011) Netflow datasets for machine learning-based network intrusion detection systems. *arXiv: 2011.09144*. <https://doi.org/10.48550/arXiv.2011.09144>
- Sarhan M et al. (2020) Netflow datasets for machine learning-based network intrusion detection systems. In: *Big data technologies and applications*. Springer, Cham, pp 117–135. https://doi.org/10.1007/978-3-030-72802-1_9

- Shuaiyi L, Wang K, Zhang L et al (2023) Process-Oriented heterogeneous graph learning in GNN-Based ICS anomalous pattern recognition. *Pattern Recogn* 141:109661. <https://doi.org/10.1016/j.patcog.2023.109661>
- Sun, P, Guo Z, Wang J, Li J, Lan J, Hu Y (2021) Deepweave: accelerating job completion time with deep reinforcement learning-based coflow scheduling. In: *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*, pp 3314–3320. <https://www.ijcai.org/proceedings/2020/0458.pdf>
- Veličković P et al. (2017) Graph attention networks. *arXiv preprint arXiv:1710.10903*. <https://doi.org/10.48550/arXiv.1710.10903>
- Vormayr G, Zseby T, Fabini J (2017) Botnet communication patterns. *IEEE Commun Surv Tutor* 19(4):2768–2796. <https://doi.org/10.1109/COMST.2017.2749442>
- Willinger W, Taqqu MS, Wilson DV (2019) Lessons from "on the self-similar nature of ethernet traffic". *ACM SIGCOMM Comput Commun Rev* 49(5):56–62. <https://doi.org/10.1145/3371934.3371955>
- Yu Z (2014) The detection and tracing of DDoS attack based on the analysis of self-similarity behaviors. Central South University, MA thesis
- Zeng Z, Peng W, Zeng D (2022) Improving the stability of intrusion detection with causal deep learning. *IEEE Trans Netw Serv Manage* 19(4):4750–4763. <https://doi.org/10.1109/TNSM.2022.3193099>
- Zhang XM, Liang L, Liu L et al (2021a) Graph neural networks and their current applications in bioinformatics. *Front Genet* 12:690049. <https://doi.org/10.3389/fgene.2021.690049>
- Zhang C, Chen Y, Meng Y et al (2021b) A novel framework design of network intrusion detection based on machine learning techniques. *Secur Commun Netw* 2021(1):6610675
- Zhou X, Liang W, Li W et al (2021) Hierarchical adversarial attacks against graph-neural-network-based IoT network intrusion detection system. *IEEE Internet Things J* 9(12):9310–9319. <https://doi.org/10.1109/JIOT.2021.3130434>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.